



Dissertation on

“Generative Clinical Narrative and Risk Explanation from ICU Time-Series Data Using Large Language Models”

M.Tech Final Semester Project (4th Sem)
Using eICU Collaborative Research Database

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in

Data Science & Artificial Intelligence

UE19CS651 – Project Phase - 2

Submitted by:
Surya Pratap Singh Shekhawat
PES2PGE24DS117

Under the guidance of
Mahesh RAMEGOWDA
Prof., PES University

January – May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru 560 100, Karnataka, India

Certificate

This is to certify that the dissertation entitled “Generative Clinical Narrative and Risk Explanation from ICU Time-Series Data Using Large Language Models” is a bonafide work carried out by **Surya Pratap Singh Shekhawat (PES2PGE24DS117)** in partial fulfilment for the completion of Fourth Semester Project Phase - 2 (UE19CS651) in the Program of Study — Master of Technology in Data Science & Artificial Intelligence under the rules and regulations of PES University, Bengaluru during the period Jan. 2026 – May 2026.

Mahesh RAMEGOWDA
Prof. CSE

Declaration

We hereby declare that the Project Phase - 2 entitled “Generative Clinical Narrative and Risk Explanation from ICU Time-Series Data Using Large Language Models” has been carried out under the guidance of Mahesh RAMEGOWDA, PES University, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science & Artificial Intelligence of PES University, Bengaluru during the academic semester January – May 2026.

Acknowledgement

I would like to express my gratitude to Mahesh RAMEGOWDA, Department of Computer Science and Engineering, PES University, for his continuous guidance and support. I am also thankful to the project coordinators for their help throughout the process. I take this opportunity to thank the Chairperson of the Department of CSE, PES University, for the support and guidance. I am grateful to Dr. B.K. Keshavan, Dean of Faculty, and the university leadership for the opportunities provided. Finally, I extend heartfelt thanks to my family and friends for their constant support.

Abstract

This dissertation presents a system for **generative clinical narrative and risk explanation** from ICU time-series data using large language models, grounded in the **eICU Collaborative Research Database** (eICU-CRD). The work transforms structured critical-care measurements—vitals, laboratory results, respiratory support, and derived risk scores—into natural-language summaries that improve interpretability and support clinical decision-making.

The implementation loads the PhysioNet **eICU-CRD demo v2.0.1** (2,500+ ICU stays, SQLite and CSV.gz tables) into `data/eicu-crd-demo/` and maps real `patientunitstayid` records to the `icu_agents` pipeline. Specialized agents (**RiskAgent**, **NarrativeAgent**, with synthetic fallbacks for sparse modalities) produce coherent patient bundles orchestrated by **PatientPipeline** and `data/pipeline_source.py`. Apache Airflow DAGs apply **rule-based clinical reasoning**—hourly severity labelling, hypoxia/sepsis branching, and coordinator escalation—on eICU-derived vitals and lactate panels.

A **Streamlit clinical dashboard** provides **point-in-time ICU snapshots**: clinicians scrub an ICU hour slider (or click interactive Plotly charts) to view vitals, labs, events, and narratives known only up to that hour—avoiding future-data leakage. A **Critical Patient Brief** module (`clinical/critical_brief.py`) generates SBAR-style summaries, six-hour trend deltas, explainable alerts with evidence, and rule-based recommended actions. **Doctor dialogue** (**ClinicalChatAgent**) accepts clinician feedback; a **feedback overlay** (`clinical/feedback_overlay.py`) adjusts trajectory, alerts, and actions while preserving underlying measurements. OpenAI LLM integration (LangChain) enriches dialogue when an API key is configured.

Contents

1	Introduction	10
1.1	Problem Statement	10
1.2	Objectives	10
1.3	Scope of the Project	11
1.4	Organization of the Report	11
2	Literature Survey	12
2.1	Existing Systems	12
2.2	Limitations of Existing Systems	12
2.3	Proposed Solution	12
3	System Requirements Specification	14
3.1	Hardware Requirements	14
3.2	Software Requirements	14
3.3	Functional Requirements	14
3.4	Non-Functional Requirements	15
3.5	Use Case Diagram	15
4	Proposed Methodology	16
4.1	System Architecture	16
4.2	Algorithm or Model Description	17
4.3	Workflow Diagram	18
5	Implementation Details	19
5.1	Development Environment	19
5.2	Dataset Description	19
5.3	Model Implementation	20
5.4	Evaluation Metrics	20
6	Results and Discussion	22
6.1	Result Analysis	22
6.2	Streamlit Dashboard Screenshots	22
7	Conclusion and Future Work	29
7.1	Conclusion	29
7.2	Future Work	29

List of Figures

4.1	System Architecture Diagram (multi-agent orchestrator, agents, Airflow, and output stores)	17
6.1	Sidebar pipeline setup: data source selection (eicu/synthetic), patient unitstayid picker, and Run ICU Pipeline trigger that loads vitals, labs, notes, and respiratory data from the eICU-CRD demo SQLite database. .	23
6.2	Critical Patient Brief: automated SBAR summary with six-hour trend deltas (SpO ₂ , heart rate, lactate, creatinine, FiO ₂), active alert count, assessment/recommendation text, prioritized actions, and data-gap warnings for the selected ICU hour.	23
6.3	Point-in-time temporal navigation: jump buttons (admission, mid-stay, latest), ICU hour slider, and chart click-to-scrub controls. All dashboard content reflects data known only up to the selected hour.	24
6.4	Overview tab: consolidated bedside snapshot with multi-series vitals charts, temperature trend, vitals table, laboratory bar chart, and clinical events timeline at the scrubbed hour.	24
6.5	Vitals (time series) tab: detailed Plotly chart of heart rate, SpO ₂ , respiratory rate, temperature, and systolic BP through the selected hour, with current-hour marker and expandable JSON history.	25
6.6	Labs & risk tab: lab draw events over time, latest laboratory panel, heuristic mortality/sepsis/deterioration risk scores, respiratory support meta-data, and radiology/notes JSON.	25
6.7	Clinical narrative tab: NarrativeAgent point-in-time report incorporating demographics, severity, vitals/labs, and recent events, with an events timeline chart below the text.	26
6.8	Doctor dialogue tab: ClinicalChatAgent conversational interface with quick prompts (severity rationale, next steps, clinical disagreement), chat history per patient/hour, and OpenAI or rule-based responses.	26
6.9	Clinician feedback overlay: after disagreeing with deterioration, the impact panel shows trajectory revision (STABLE → IMPROVING), interpreted feedback notes, clinician-directed actions, and a clinician-adjusted Critical Patient Brief with automated baseline available for comparison.	28

List of Tables

5.1	Implementation module map	20
-----	-------------------------------------	----

Chapter 1

Introduction

Critical care environments generate high-frequency, multimodal data: vital signs, laboratory results, imaging reports, ventilator settings, and risk assessments. The **eICU Collaborative Research Database** provides a widely used, de-identified resource for ICU research, while recent large language models (LLMs) enable transformation of structured time-series into readable clinical narratives with temporal reasoning.

This M.Tech final-semester project (4th semester) addresses the gap between raw ICU tables and clinician-facing explanations. The `icu_agents` module loads the PhysioNet eICU-CRD demo from local SQLite, maps vitals and laboratories to a shared **PatientS tate** schema, and runs rule-based risk scoring plus narrative generation. Apache Airflow DAGs apply hourly severity reasoning and hypoxia/sepsis branching on real time-series, with synthetic fallbacks when charting is sparse.

The Streamlit interface supports **point-in-time clinical review**: all dashboard tabs (Overview, Vitals, Labs & Risk, Clinical Narrative, Doctor Dialogue) reflect patient state at a selected ICU hour. Interactive Plotly charts allow click-to-scrub timeline navigation. A **Critical Patient Brief** presents SBAR summaries, six-hour trend deltas, explainable alerts, and escalation actions. Clinician dialogue can revise the automated assessment—trajectory, alerts, and recommendations—via a structured feedback overlay, demonstrating a human-in-the-loop decision-support workflow. Code is maintained at <https://github.com/suryashekhawat/MTech4SEM>.

1.1 Problem Statement

Clinicians and researchers struggle to interpret high-dimensional ICU time-series at the bedside. Structured vitals and labs in relational tables are difficult to scan quickly, while risk scores alone lack narrative context. The problem addressed is to **transform structured ICU data into natural-language summaries**, apply **LLMs for clinical narrative generation and risk explanation**, and improve **interpretability and decision support**—using eICU-CRD where available and a modular multi-agent pipeline for end-to-end prototyping.

1.2 Objectives

The primary objectives of the project are:

- To transform structured ICU time-series from eICU-CRD into natural-language clinical summaries.
- To apply large language models for narrative generation, risk explanation, and temporal reasoning.
- To build a Streamlit UI with **point-in-time ICU snapshots**, interactive time-series charts, and tabbed clinical views (Overview, Vitals, Labs & Risk, Clinical Narrative, Doctor Dialogue).
- To design a multi-agent architecture (`icu_agents`) for vitals, labs, radiology, respiratory, risk, and narrative outputs.
- To integrate Apache Airflow DAGs for batch generation, temporal branching, and artifact persistence.
- To deliver a **Critical Patient Brief** (SBAR, trend deltas, explainable alerts, recommended actions) for critical-care decision support.
- To support **clinician dialogue and feedback overlay**, adjusting trajectory and recommendations while preserving factual vitals/labs.
- To improve interpretability and prototype clinical decision-support workflows (OpenAI-backed dialogue, future BeeAI deployment).

1.3 Scope of the Project

The scope includes:

- eICU-CRD demo access (<https://physionet.org/content/eicu-crd-demo/2.0.1/>), automated download via `scripts/download_eicu_crd_demo.py`, and SQLite-backed loading in `icu_agents/data/eicu_loader.py`.
- Dual-mode execution (`ICU_DATA_SOURCE=eicu` or `synthetic`); Streamlit and CLI entry points support stay selection by `patientunitstayid`.
- Hour-level temporal scrubbing via `data/temporal_timeline.py` (`build_timeline`, `snapshot_at_hour`, `patient_view_at_hour`), interactive Plotly charts (`ui/chart_helpers.py`), the Critical Patient Brief (`clinical/critical_brief.py`), and clinician feedback overlay (`clinical/feedback_overlay.py`).
- Airflow DAGs mount the local demo dataset and persist JSON/report/timeline artifacts under `icu_agents/output/`.
- LLM integration supports narrative enrichment and conversational doctor dialogue when `OPENAI_API_KEY` is set.

The system is a research prototype and not validated for live clinical deployment.

1.4 Organization of the Report

Chapter 1 introduces the problem, objectives, and scope. Chapter 2 surveys related work and positions the proposed multi-agent approach. Chapter 3 specifies hardware, software, and functional requirements. Chapter 4 describes methodology, architecture, and workflows. Chapter 5 details implementation modules and dependencies. Chapter 6 discusses results and evaluation considerations. Chapter 7 concludes and outlines future enhancements.

Chapter 2

Literature Survey

Healthcare analytics increasingly relies on synthetic and semi-synthetic data when real cohorts are unavailable. Prior work on ICU scoring (e.g., severity indices) and sepsis detection emphasizes temporal vitals and laboratory markers such as lactate. Workflow engines such as Apache Airflow are widely used for ETL and ML pipelines in industry. Multi-agent and agentic AI frameworks (including LangChain and LangGraph, listed in project dependencies) support decomposed reasoning tasks, though this project’s agents are currently implemented as plain Python classes with explicit orchestration.

2.1 Existing Systems

Existing approaches to ICU data simulation include:

- **Statistical simulators** that sample marginals independently, often breaking cross-field clinical consistency.
- **EHR-based synthetic generators** (e.g., GAN or copula-based) that require source data and substantial training infrastructure.
- **Rule-based clinical engines** embedded in research simulators, which encode domain knowledge but may lack modular orchestration.
- **Workflow-orchestrated pipelines** in production ML platforms, which schedule tasks but may not model clinical agents explicitly.

2.2 Limitations of Existing Systems

Monolithic generators are difficult to extend when new modalities (e.g., radiology narratives) are required. Black-box deep generative models demand large datasets and offer limited interpretability for education and pipeline testing. Systems without orchestration layers complicate scheduling, artifact versioning, and audit trails. Few lightweight prototypes combine interactive UI, batch DAG execution, and temporal branching for escalation logic in a single codebase.

2.3 Proposed Solution

The proposed `icu_agents` system addresses these gaps through:

1. **Specialized agents** (VitalsAgent, LabAgent, RadiologyAgent, RespiratoryAgent, RiskAgent, NarrativeAgent) with diagnosis-aware parameters.
2. **A unified PatientState** model aggregating all modalities.
3. **PatientPipeline** for in-process orchestration and **Airflow DAGs** for production-style task graphs.
4. **Temporal reasoning** with hourly severity labels, point-in-time snapshots, and branch tasks for respiratory and sepsis escalation paths.
5. **Critical Patient Brief** with SBAR formatting, six-hour trend deltas, explainable alerts, and rule-based action recommendations.
6. **Human-in-the-loop feedback** via doctor dialogue and a clinician-adjusted brief overlay.
7. **Persistent artifacts** (JSON bundles, text reports, timeline traces, dashboard snapshot).

Chapter 3

System Requirements Specification

This chapter covers the system requirements, including hardware and software specifications necessary for the project.

3.1 Hardware Requirements

- 64-bit processor (Intel/AMD x64 or Apple Silicon with emulation).
- Minimum 8 GB RAM (16 GB recommended for Docker-based Airflow with PostgreSQL).
- 10 GB free disk space for Python environment, Docker images, and generated output/ artifacts.
- Network access for initial dependency installation (pip, Docker image pulls).

3.2 Software Requirements

- **Operating system:** Windows 10/11 (app and Streamlit), WSL2 Ubuntu, or Linux/macOS for full Airflow support.
- **Python:** 3.10+ with `venv`.
- **Core libraries:** Pydantic, Pandas, NumPy, Faker, Streamlit, Matplotlib, Plotly, scikit-learn, NetworkX, LangChain (see `requirements.txt`).
- **Orchestration (optional):** Apache Airflow 2.x via Docker Compose (PostgreSQL 15 backend) or WSL2-native install.
- **Container runtime:** Docker Desktop (recommended for Airflow on Windows).
- **Version control:** Git.

3.3 Functional Requirements

1. **FR1:** Generate a synthetic patient with UUID, age, gender, and one of six ICU diagnoses.
2. **FR2:** Produce 24-hour vital sign time series (heart rate, temperature, SpO₂, respiratory rate, systolic BP) with diagnosis-specific baselines.
3. **FR3:** Generate laboratory results (WBC, lactate, creatinine, platelets, hemoglobin).
4. **FR4:** Attach radiology report text mapped to diagnosis.
5. **FR5:** Set respiratory support fields (mechanical ventilation flag, FiO₂, PEEP).

6. **FR6:** Compute mortality, sepsis, and ICU deterioration risk scores from latest vitals and labs.
7. **FR7:** Render a human-readable ICU narrative summary.
8. **FR8:** Persist patient bundles and reports via Airflow tasks to `output/json` and `output/reports`.
9. **FR9:** Update `latest_dashboard.json` with key metrics after batch runs.
10. **FR10:** Execute temporal pipeline with hourly severity trace, hypoxia/sepsis branching, and coordinator recommendations.
11. **FR11:** Provide Streamlit UI to trigger generation and display narrative, vitals, labs, and risk JSON.
12. **FR12:** Support **point-in-time ICU review** via an hour slider and `snapshot_at_hour()` so all UI tabs reflect data known only up to the selected hour.
13. **FR13:** Render **interactive Plotly charts** (vitals, labs-over-time, clinical events) with click-to-scrub timeline navigation.
14. **FR14:** Generate a **Critical Patient Brief** (SBAR, six-hour trend deltas, explainable alerts with evidence, recommended actions, data-gap warnings).
15. **FR15:** Enable **doctor dialogue** (`ClinicalChatAgent`) with optional OpenAI-backed responses and rule-based fallback.
16. **FR16:** Apply **clinician feedback overlay** to adjust trajectory, alerts, and actions while preserving underlying measurements; display before/after impact.

3.4 Non-Functional Requirements

- **Modularity:** Each agent resides in a separate module under `agents/`.
- **Reproducibility:** Configurable random seed (`SEED=42` in `config.py`) for future deterministic runs.
- **Extensibility:** New diagnoses can be added to `clinical_rules.DIAGNOSES` and agent conditional logic.
- **Portability:** Docker Compose standardizes Airflow deployment across machines.
- **Usability:** Streamlit offers one-click patient generation for demonstrations.
- **Maintainability:** Pydantic models validate patient structure; Airflow task boundaries mirror agent boundaries.

3.5 Use Case Diagram

Primary actors and use cases include: **Clinician/Researcher** → run pipeline, scrub ICU hour, review Critical Patient Brief, provide doctor dialogue feedback; **Operator** → run `app.py`, trigger Streamlit, or start Airflow DAGs; **Scheduler** → daily execution of temporal and multi-agent DAGs; **System** → write JSON/report/timeline artifacts, apply feedback overlay, and update dashboard metadata.

Chapter 4

Proposed Methodology

This chapter explains the overall methodology, system architecture, data flow, and algorithms used in the `icu_agents` implementation.

4.1 System Architecture

The architecture has five layers:

1. **Data:** PhysioNet eICU-CRD demo SQLite (`eicu_v2_0_1.sqlite3`) and CSV.gz tables under `data/eicu-crd-demo/`, loaded by `data/eicu_loader.py`.
2. **Presentation:** `app.py` (CLI) and `ui/streamlit_app.py` (web UI with stay selector, ICU hour scrubber, Critical Patient Brief, doctor dialogue).
3. **Clinical decision support:**
 - `clinical/critical_brief.py` — SBAR, trends, alerts, actions
 - `clinical/feedback_overlay.py` — clinician-adjusted brief
 - `data/temporal_timeline.py` — point-in-time snapshots
4. **Orchestration:** `orchestrator/patient_pipeline.py` and `data/pipeline_source.py`; Apache Airflow DAGs for batch and temporal reasoning.
5. **Agent layer:** Risk and narrative agents on real measurements; synthetic generators used only when eICU fields are missing.

eICU data path. For `ICU_DATA_SOURCE=eicu`, the loader queries core eICU tables including `patient`, `vitalperiodic`, `lab`, `admissiondx`, `diagnosis`, `note`, and `respiratorycharting` for a selected `patientunitstayid`. Vitals are mapped to hourly traces (heart rate, SpO_2 , respiratory rate, temperature, systolic BP); labs are normalized to WBC, lactate, creatinine, platelets, and hemoglobin. `RiskAgent` and `NarrativeAgent` consume this `PatientState`; sparse radiology rows fall back to `RadiologyAgent` templates.

Synthetic fallback path. When `ICU_DATA_SOURCE=synthetic`, `generate_base_patient()` seeds demographics and all modality agents run as before.

Airflow DAGs call `build_patient_dict()` so the same eICU mapping is used in Docker (volume-mounted demo data) and on the host.

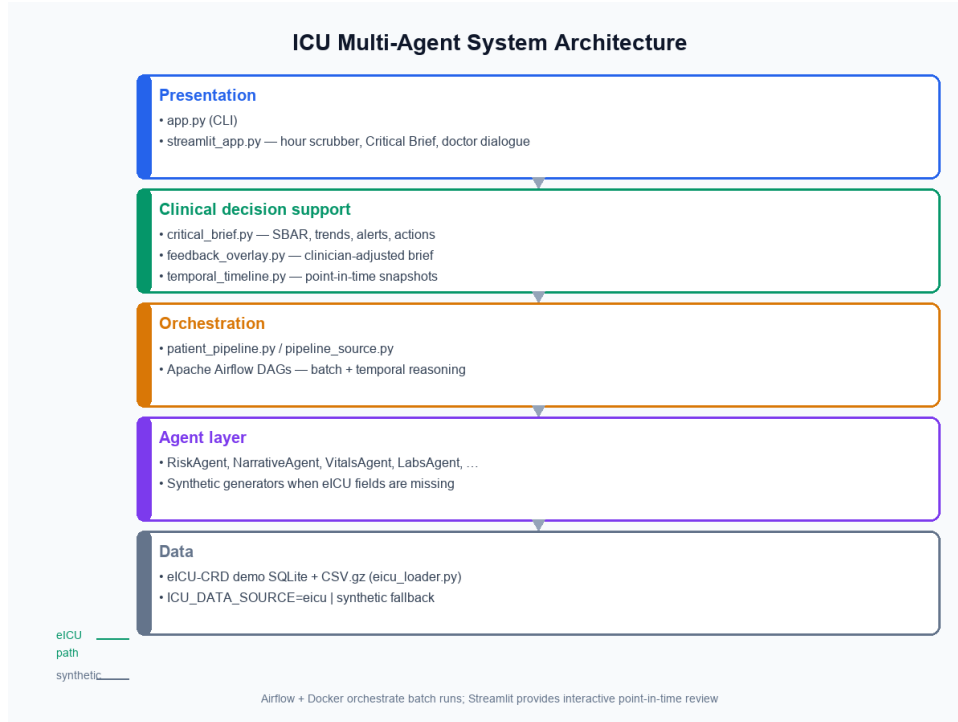


Figure 4.1: System Architecture Diagram (multi-agent orchestrator, agents, Airflow, and output stores)

4.2 Algorithm or Model Description

Base patient generation. `generate_base_patient()` randomly selects age (24–88), gender (Male/Female), and diagnosis from six conditions.

VitalsAgent. For each hour in a 24-hour window, samples vitals from Gaussian distributions centered on diagnosis-specific baselines (e.g., Sepsis: higher heart rate and temperature, lower SpO₂; ARDS: SpO₂ baseline 85%).

LabAgent. Samples WBC, lactate, creatinine, platelets, and hemoglobin with diagnosis-adjusted means (elevated lactate and WBC for Sepsis).

RadiologyAgent. Returns a fixed impression string per diagnosis from a lookup table.

RespiratoryAgent. Sets mechanical ventilation and FiO₂/PEEP ranges for ARDS and Sepsis; otherwise room-air defaults.

RiskAgent. Rule-based scoring: adds weighted points if SpO₂ < 90, heart rate > 120, lactate > 4, or creatinine > 2; caps mortality risk at 100 and derives correlated sepsis and deterioration scores.

Temporal timeline and point-in-time snapshots. `build_timeline()` merges hourly vitals, lab draws, clinical notes, and respiratory events from eICU SQLite. `snapshot_at_hour()` reconstructs cumulative labs, ventilation settings, risk scores, and events known only up to the selected ICU hour—preventing future-data leakage in narratives and charts. `NarrativeAgent.generate(hour=..., recent_events=...)` produces hour-specific clinical reports.

Critical Patient Brief. `build_critical_brief()` computes six-hour trend deltas (SpO₂, heart rate, lactate, creatinine, FiO₂), classifies overall trajectory (deteriorating/improving/stable), emits explainable alerts (hypoxemia, tachycardia, hyperlac-

tatemia, AKI) with hour-stamped evidence, and recommends rule-based actions aligned with Airflow escalation logic (respiratory and sepsis pathways).

Clinician feedback overlay. `apply_doctor_feedback()` parses doctor dialogue (agreement, disagreement, improving/worsening cues, plan keywords) and adjusts the brief—trajectory label, alert set, SBAR assessment/recommendation, and action list—while leaving vitals and labs unchanged. The UI displays a before/after impact panel and an automated baseline expander for comparison.

Interactive dashboard. Plotly charts (`ui/chart_helpers.py`) support hover tooltips, selected-hour markers, and click-to-scrub navigation synchronized across tabs via a shared `pit_hour_slider` session state.

Temporal reasoning (Airflow, eICU-backed). `build_hourly_reasoning_trace` walks the eICU-derived vitals list and labels each hour as *stable*, *worsening*, or *critical* when SpO_2 or heart rate cross clinical thresholds (e.g., $\text{SpO}_2 < 90$ and $\text{HR} > 115 \Rightarrow$ worsening; $\text{SpO}_2 < 85$ or $\text{HR} > 125 \Rightarrow$ critical). Each trace row stores an explicit `reason` string for auditability.

Branch tasks `branch_hypoxia` and `branch_septic_shock` route to escalation when terminal $\text{SpO}_2 < 85$ or lactate > 4 mmol/L from the eICU lab panel. `coordinator_agent` merges branch outcomes, proposes FiO_2/PEEP adjustments, and emits sepsis action lists; `generate_temporal_narrative` summarizes critical hours and escalation flags for the final report.

4.3 Workflow Diagram

Local pipeline (`PatientPipeline.run`):

1. Generate base patient $\rightarrow$ create `PatientState` with UUID.
2. `VitalsAgent` $\rightarrow$ `LabAgent` $\rightarrow$ `RadiologyAgent` $\rightarrow$ `RespiratoryAgent`.
3. `RiskAgent` $\rightarrow$ `NarrativeAgent` $\rightarrow$ return patient.

Airflow `icu_multi_agent_pipeline` (eICU-integrated):

1. `load_patient_from_source` (eICU SQLite $\rightarrow$ risk $\rightarrow$ narrative)
2. `store_patient_bundle` $\rightarrow$ `update_dashboard`

Streamlit point-in-time dashboard:

1. Load eICU stay $\rightarrow$ `build_timeline()`.
2. Clinician selects ICU hour (slider or chart click) $\rightarrow$ `snapshot_at_hour()`.
3. `build_critical_brief()` $\rightarrow$ SBAR, trends, alerts, actions.
4. Optional doctor dialogue $\rightarrow$ `ClinicalChatAgent` $\rightarrow$ `apply_doctor_feedback()` $\rightarrow$ clinician-adjusted brief and impact panel.
5. All tabs refresh: Overview, Vitals, Labs & Risk, Clinical Narrative, Doctor Dialogue.

Airflow `icu_temporal_reasoning_pipeline`:

1. `load_temporal_bundle` (eICU vitals/labs)
2. `build_hourly_reasoning_trace`
3. Parallel branch tasks (hypoxia, sepsis)
4. `coordinator_agent` $\rightarrow$ `optional_langgraph_handoff`
5. `generate_temporal_narrative` $\rightarrow$ `store_temporal_outputs`

Chapter 5

Implementation Details

This chapter provides details of the development process, programming languages used, tools and libraries, module layout, and evaluation approach.

5.1 Development Environment

Development was carried out in Python using a virtual environment (`python-mvenv.venv`). The package root is `icu_agents/`. Local execution:

```
cd icu_agents
pip install -r requirements.txt
python app.py
streamlit run ui/streamlit_app.py
```

Airflow is deployed via `docker-compose.yml` (PostgreSQL, webserver on port 8080, scheduler) with DAGs mounted from `airflow/dags/` and the eICU demo volume `../data/eicu-crd-demo`.

Project configuration in `config.py` defines `DATA_SOURCE` (default `eicu`), `EICU_DEMO_DIR`, `OPENAI_MODEL`, `MAX_ICU_HOURS=72`, and `SEED=42`. Download the dataset with `scripts/download_eicu_crd_demo.sh` before first run.

5.2 Dataset Description

eICU Collaborative Research Database (demo v2.0.1). The primary data source is the open PhysioNet demo (<https://physionet.org/content/eicu-crd-demo/2.0.1/>), comprising 2,500+ de-identified ICU stays from 20 hospitals. Files are stored locally under `data/eicu-crd-demo/` (git-ignored) and include gzipped CSV tables plus `sqlite/eicu_v2_0_1.sqlite3`.

The loader maps `vitalperiodic` rows to hourly traces, latest `lab` values to the agent lab schema, `note` text to radiology reports when available, and `respiratorycharting` FiO_2 /PEEP to ventilation fields.

Agent bundles (eICU + synthetic fallback). Each bundle includes:

- **Demographics:** `patient_id`, `age`, `gender`, `diagnosis`.
- **vitals:** list of 24 hourly dictionaries (`timestamp`, `heart_rate`, `temperature`, `spo2`, `resp_rate`, `systolic_bp`).

- **labs:** WBC, Lactate, Creatinine, Platelets, Hemoglobin.
- **radiology:** report text; **respiratory:** ventilation parameters.
- **risk_scores:** mortality_risk, sepsis_risk, icu_deterioration_risk.
- **narrative:** formatted clinical summary string.
- **data_source:** eicu or synthetic; eicu_stay_id when loaded from the demo database.

Temporal runs add `reasoning_trace` (per-hour severity and reason), `respiratory_escalation`, `sepsis_escalation`, `coordinator_summary`, and `temporal_narrative`. Artifacts are written to `output/json`, `output/reports`, and `output/timelines`.

5.3 Model Implementation

Key modules and responsibilities:

Module	Role
<code>models/patient_state.py</code>	Pydantic <code>PatientState</code> schema
<code>data/eicu_loader.py</code>	eICU SQLite queries and schema mapping
<code>data/pipeline_source.py</code>	Unified eICU/synthetic bundle builder
<code>synthetic/clinical_rules.py</code>	Diagnosis list and base patient sampling
<code>agents/vitals_agent.py</code>	Hourly vitals time series
<code>agents/lab_agent.py</code>	Laboratory panel generation
<code>agents/radiology_agent.py</code>	Imaging report lookup
<code>agents/respiratory_agent.py</code>	Ventilation settings
<code>agents/risk_agent.py</code>	Rule-based risk scoring
<code>data/temporal_timeline.py</code>	Point-in-time timeline, snapshots, hour views
<code>clinical/critical_brief.py</code>	SBAR brief, trends, alerts, actions
<code>clinical/feedback_overlay.py</code>	Clinician feedback impact on brief
<code>agents/clinical_chat_agent.py</code>	Doctor dialogue (LLM + rule fallback)
<code>ui/chart_helpers.py</code>	Interactive Plotly charts for scrubbing
<code>agents/narrative_agent.py</code>	Point-in-time and full-stay narratives
<code>orchestrator/patient_pipeline.py</code>	Agent sequencing
<code>airflow/dags/icu_multi_agent_pipeline.py</code>	Batch multi-agent DAG
<code>airflow/dags/icu_temporal_reasoning_pipeline.py</code>	Temporal branching DAG
<code>ui/streamlit_app.py</code>	Point-in-time dashboard, brief, dialogue

Table 5.1: Implementation module map

5.4 Evaluation Metrics

Evaluation focuses on pipeline correctness, eICU mapping fidelity, and clinical plausibility of rule-based reasoning:

- **Structural validity:** JSON bundles conform to expected keys; Pydantic validation on `PatientState`.

- **eICU mapping:** `patientunitstayid` resolves in SQLite; vitals and labs populate non-empty fields for cohort stays with measurements.
- **Risk monotonicity:** On held-out synthetic cases, higher lactate and lower SpO₂ increase mortality risk; on eICU stays, scores reflect measured lactate and latest SpO₂.
- **Temporal severity:** Critical hours in `reasoning_trace` align with SpO₂/HR threshold breaches on eICU-derived traces.
- **Branching correctness:** Hypoxia branch fires when terminal SpO₂ < 85; sepsis branch when lactate > 4 mmol/L.
- **Point-in-time integrity:** Labs, vitals, and events at hour h exclude measurements with timestamp > h .
- **Brief consistency:** Alerts fire when thresholds breached; evidence hours match timeline events.
- **Feedback overlay:** Clinician “disagree + improving” downgrades trajectory and suppresses matched alerts; impact panel reflects before/after counts.
- **UI synchronization:** Chart click-to-scrub updates shared hour slider and all tab content on rerun.
- **Operational metrics:** DAG success rate, artifact creation under `output/`, dashboard metadata (`data_source`, `eicu_stay_id`).

Future work may add SOFA/APACHE calibration, distributional comparison to published ICU statistics, persisted feedback audit logs, and LLM-judged narrative quality.

Chapter 6

Results and Discussion

The pipeline was exercised through CLI runs, Streamlit sessions, and repeated Airflow DAG triggers. Each successful run produces a unique `patient_id` and persisted artifacts.

6.1 Result Analysis

Single-patient generation (eICU). Running `pythonapp.py--sourceeicu--stay-id<id>` loads a real demo stay, maps vitals/labs from SQLite, and prints the narrative. Example stay 147784 yields measured WBC, lactate, creatinine, FiO_2 from charting, and a 24-point vitals trace for reasoning.

Multi-agent Airflow DAG. With `ICU_DATA_SOURCE=eicu` and the demo volume mounted in Docker, `icu_multi_agent_pipeline` writes bundles tagged `data_source:eicu` and refreshes `latest_dashboard.json` with `eicu_stay_id`, mortality risk, and latest SpO_2 .

Temporal reasoning DAG. The `icu_temporal_reasoning_pipeline` builds `reasoning_trace` from eICU `vitalperiodic` rows, applies hypoxia/sepsis branches on measured SpO_2 and lactate, and stores coordinator escalation metadata plus `temporal_narrative` under `output/timelines/`.

Streamlit point-in-time dashboard. Operators select `patientunitstayid`, run the pipeline, and scrub ICU hours via slider or interactive Plotly charts. All tabs reflect the selected hour.

The **Critical Patient Brief** shows SBAR text, six-hour trend deltas with directional arrows, explainable alerts (e.g., tachycardia with hour-stamped evidence), and prioritized actions. The **Doctor Dialogue** tab accepts clinician feedback; the **feedback overlay** revises trajectory (e.g., STABLE $\rightarrow$ IMPROVING when the clinician disagrees with deterioration), adjusts alerts, and adds clinician-directed actions, with a before/after impact summary.

6.2 Streamlit Dashboard Screenshots

Figures 6.1–6.9 document the Streamlit clinical dashboard at `http://localhost:8501`, captured with Playwright for eICU stay 141764 at ICU hour 5. Detailed captions are maintained in `screenshots/ui_feature_captures.md`.

Discussion. Grounding agents in eICU-CRD demo data improves external validity over pure simulation while retaining interpretable rule-based reasoning traces. The

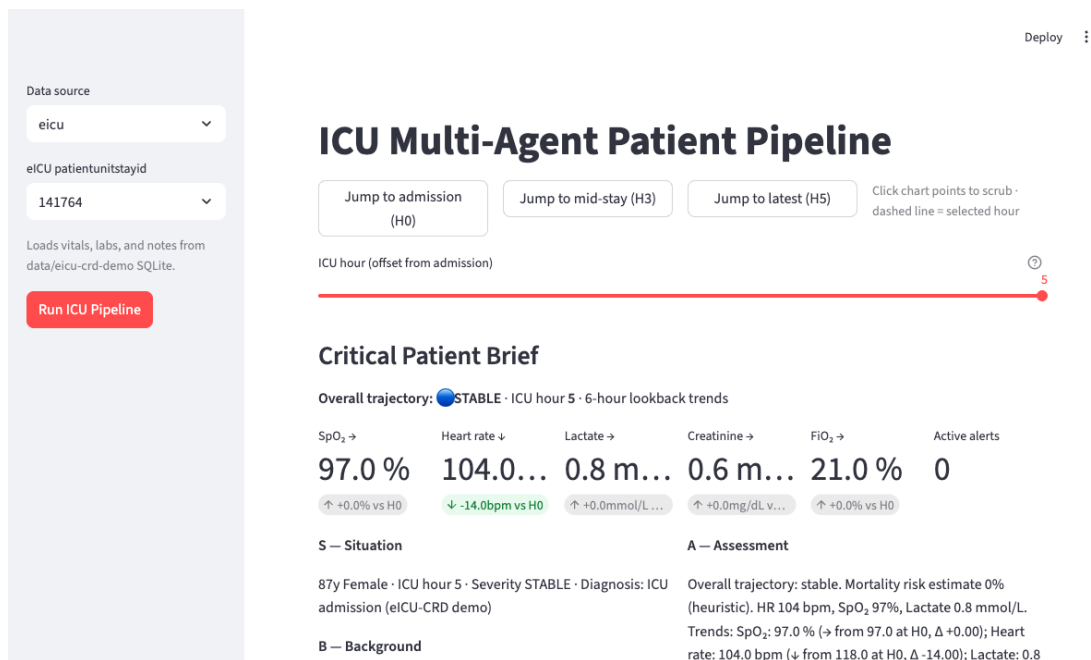


Figure 6.1: Sidebar pipeline setup: data source selection (eicu/synthetic), patientunitstayid picker, and **Run ICU Pipeline** trigger that loads vitals, labs, notes, and respiratory data from the eICU-CRD demo SQLite database.

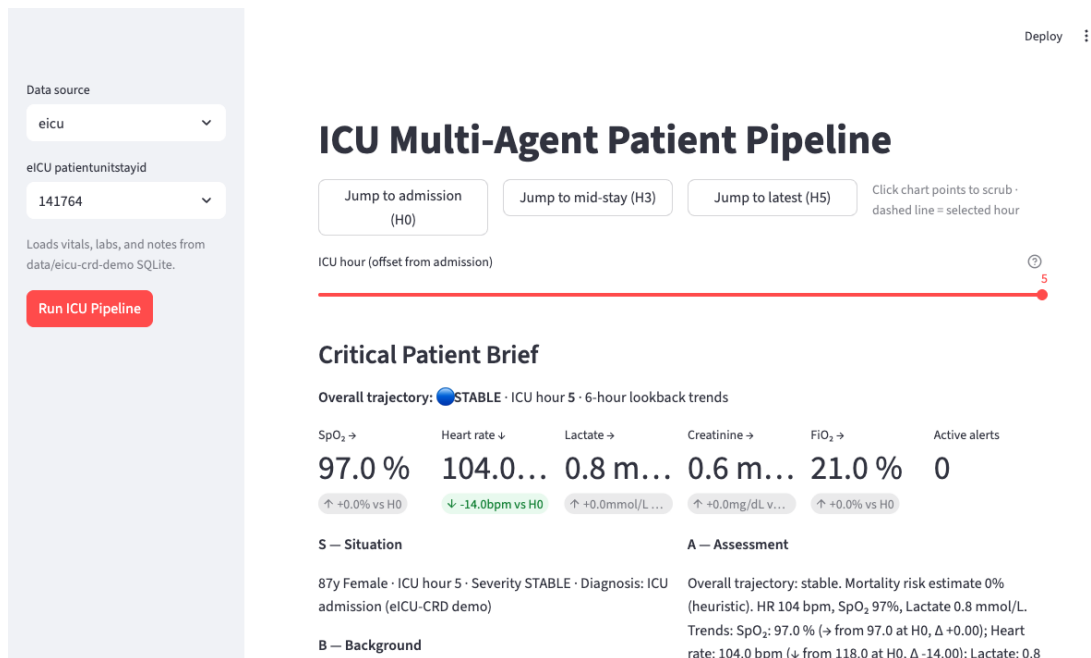


Figure 6.2: Critical Patient Brief: automated SBAR summary with six-hour trend deltas (SpO₂, heart rate, lactate, creatinine, FiO₂), active alert count, assessment/recommendation text, prioritized actions, and data-gap warnings for the selected ICU hour.

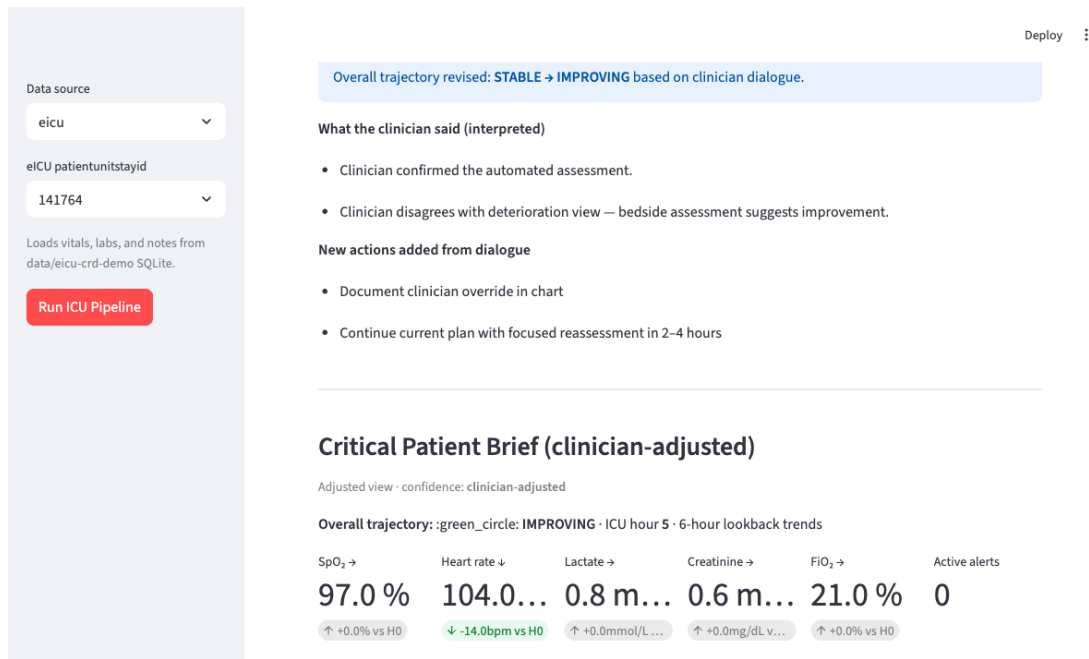


Figure 6.3: Point-in-time temporal navigation: jump buttons (admission, mid-stay, latest), ICU hour slider, and chart click-to-scrub controls. All dashboard content reflects data known only up to the selected hour.

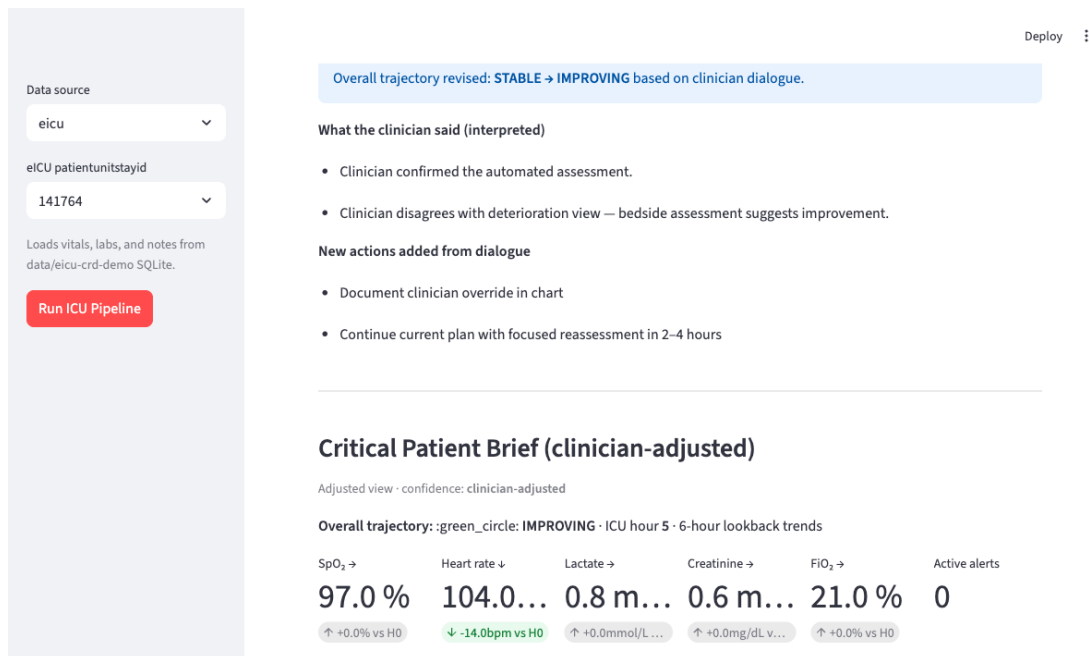


Figure 6.4: Overview tab: consolidated bedside snapshot with multi-series vitals charts, temperature trend, vitals table, laboratory bar chart, and clinical events timeline at the scrubbed hour.

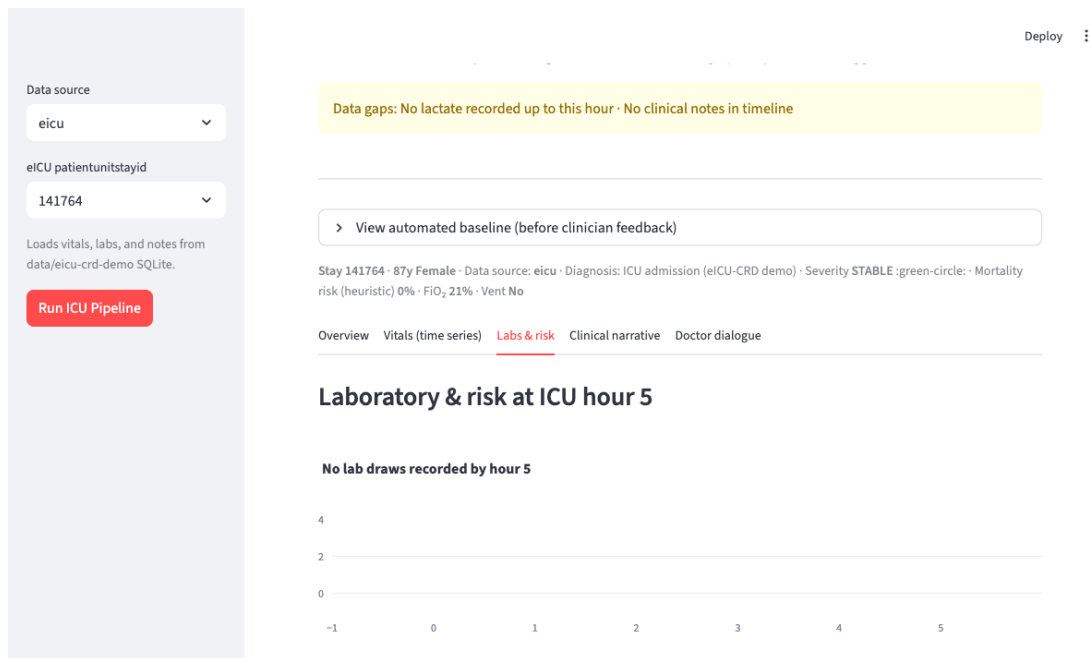


Figure 6.5: Vitals (time series) tab: detailed Plotly chart of heart rate, SpO₂, respiratory rate, temperature, and systolic BP through the selected hour, with current-hour marker and expandable JSON history.

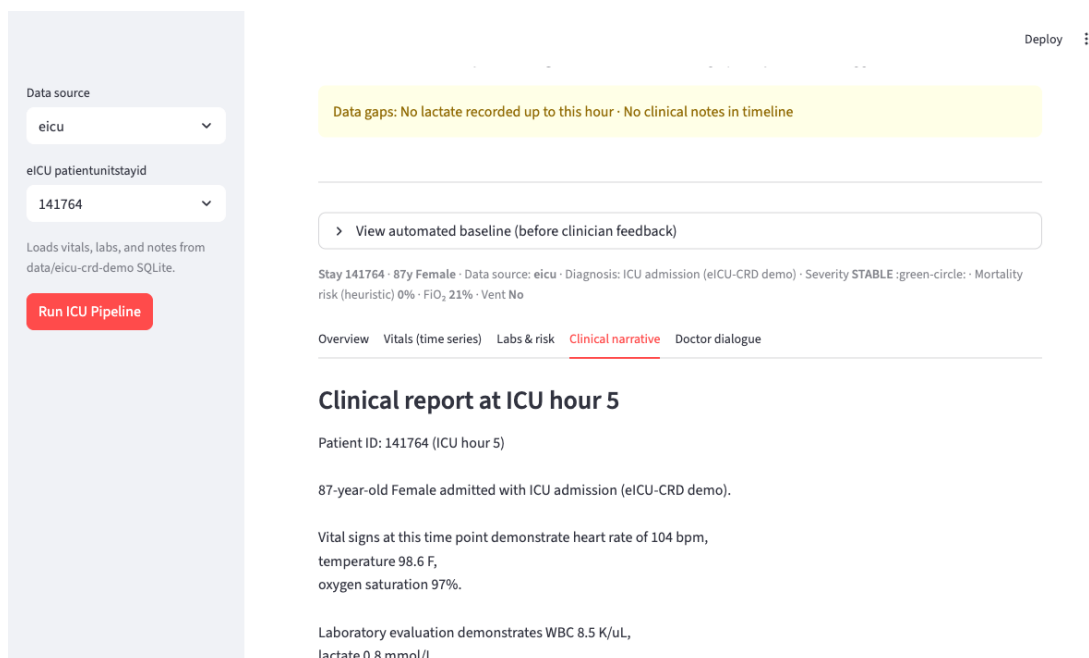


Figure 6.6: Labs & risk tab: lab draw events over time, latest laboratory panel, heuristic mortality/sepsis/deterioration risk scores, respiratory support metadata, and radiology/notes JSON.

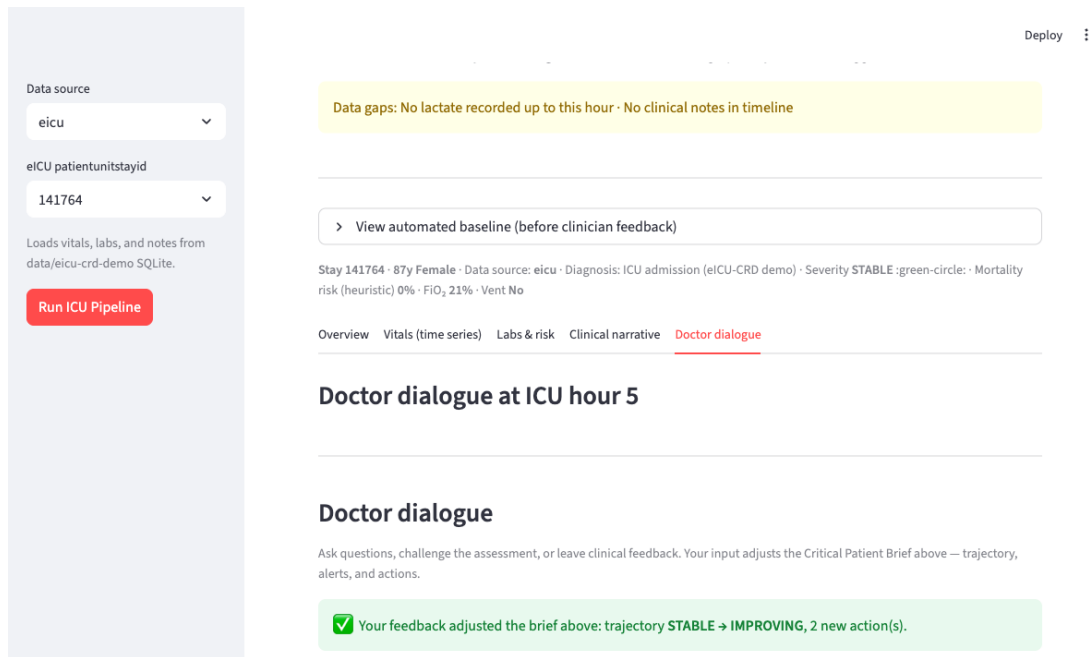


Figure 6.7: Clinical narrative tab: **NarrativeAgent** point-in-time report incorporating demographics, severity, vitals/labs, and recent events, with an events timeline chart below the text.

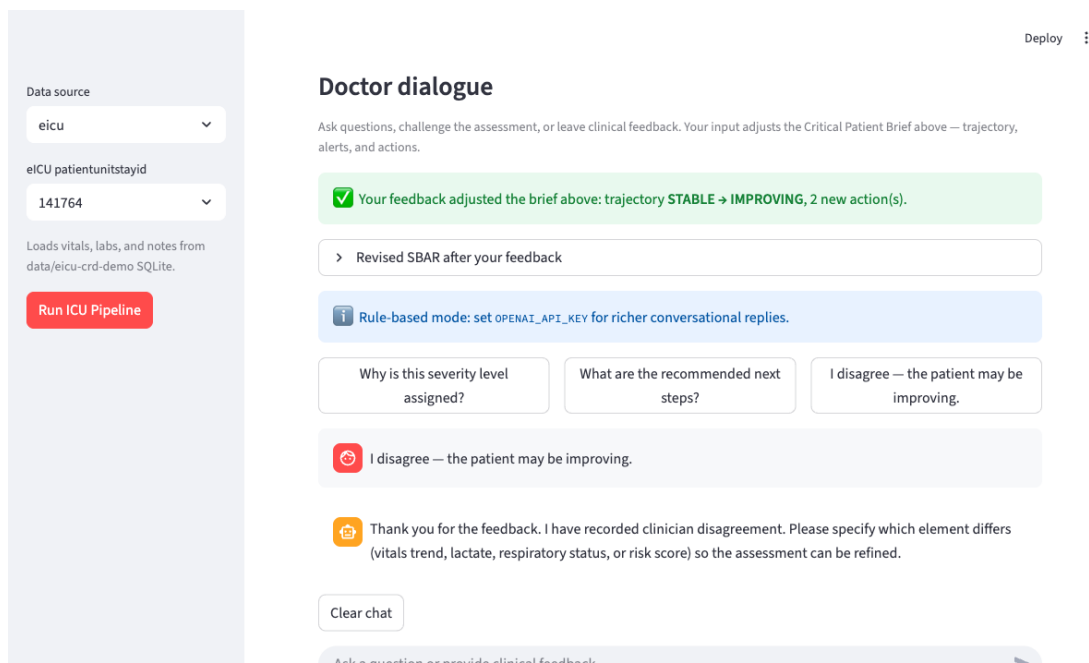


Figure 6.8: Doctor dialogue tab: **ClinicalChatAgent** conversational interface with quick prompts (severity rationale, next steps, clinical disagreement), chat history per patient/hour, and OpenAI or rule-based responses.

point-in-time architecture addresses a key clinical requirement: bedside decisions use only information available at a given moment. The feedback overlay prototypes human-in-the-loop decision support without altering factual measurements. Limitations include sparse temperature/SpO₂ in some stays, simplified risk scoring (not calibrated to APACHE/SOFA), non-persisted dialogue audit trails, and placeholder LangGraph integration. The hybrid design (eICU first, synthetic fallback) supports reproducible Airflow runs and LLM-enriched dialogue on real clinical text from **note** tables.

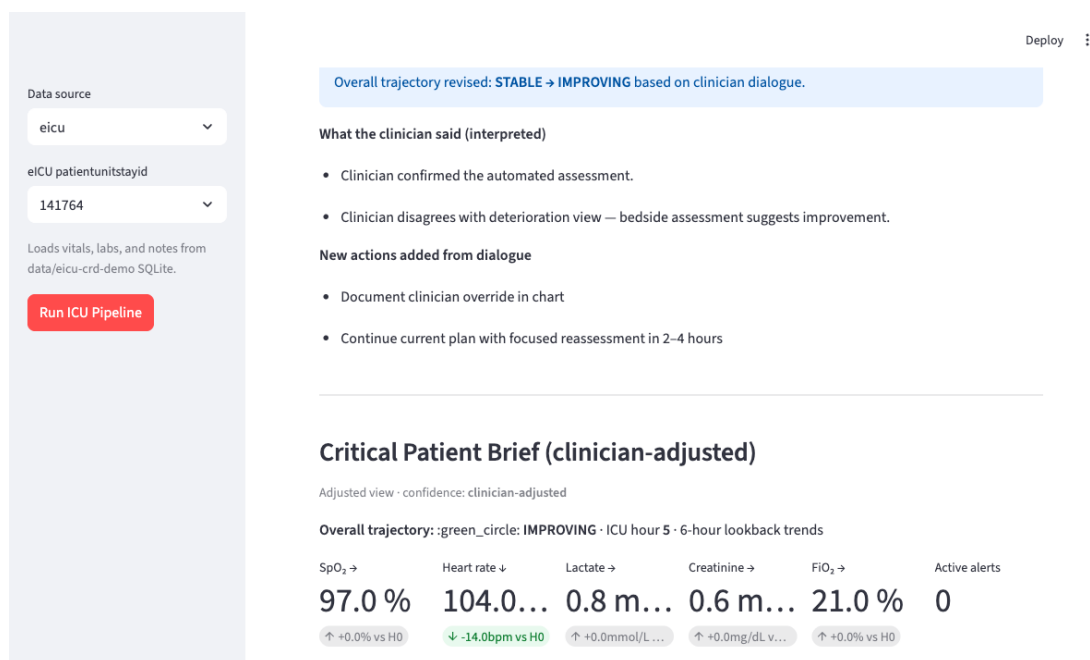


Figure 6.9: Clinician feedback overlay: after disagreeing with deterioration, the impact panel shows trajectory revision (STABLE → IMPROVING), interpreted feedback notes, clinician-directed actions, and a clinician-adjusted Critical Patient Brief with automated baseline available for comparison.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project delivered a framework for **generative clinical narrative and risk explanation** from ICU time-series data, integrating the PhysioNet eICU-CRD demo with the `icu_agents` multi-agent stack. SQLite-backed loading, stay-level orchestration, and Airflow DAGs with explicit temporal reasoning traces transform structured measurements into human-readable reports and escalation-aware narratives. The Streamlit dashboard adds **point-in-time clinical review**, a **Critical Patient Brief** (SBAR, trends, explainable alerts, actions), **interactive Plotly timeline navigation**, and a **clinician feedback overlay** that adjusts recommendations while preserving underlying data. The work establishes a foundation for LLM-enhanced, human-in-the-loop ICU decision support and aligns with the M.Tech project goals of improving interpretability through natural-language clinical summaries grounded in real critical-care data.

7.2 Future Work

- Calibrate risk scores and brief alerts against SOFA/APACHE/qSOFA using full eICU-CRD access.
- Persist clinician feedback and impact logs to `output/feedback/` for audit and model improvement.
- Multi-lane unified clinical timeline (vitals, labs, meds, notes on one scrubber).
- PDF/shift-handoff export of Critical Patient Brief at selected hour.
- Complete BeeAI agent workflow integration and Agent Stack deployment.
- Expand OpenAI LLM usage for differential reasoning and natural-language querying over eICU cohorts.
- Evaluate narratives and brief quality against clinician review and published LLM summarization benchmarks.
- Extend cohort analytics (EDA, feature stores) on top of the integrated eICU loader already shared with agents and Airflow.
- Add FHIR export and monitoring for Airflow and agent pipeline health.
- Load medication and intervention timelines from eICU `medication` and `treatment` tables.

Bibliography

- [1] Pollard, T.J., et al., “The eICU Collaborative Research Database”, *Scientific Data*, 2018. <https://physionet.org/content/eicu-crd-demo/>
- [2] “Improving Representation Learning of Complex Critical Care Data with ICU-BERT”, *arXiv:2502.19593*, 2025.
- [3] “Zero-Shot Large Language Models for Long Clinical Text Summarization with Temporal Reasoning”, *arXiv:2501.18724v2*, 2025.
- [4] Johnson, A.E.W., et al., “MIMIC-III, a freely accessible critical care database”, *Scientific Data*, 2016.
- [5] Seymour, C.W., et al., “Assessment of Clinical Criteria for Sepsis”, *JAMA*, 2016.
- [6] Ranieri, V.M., et al., “Acute Respiratory Distress Syndrome: The Berlin Definition”, *JAMA*, 2012.
- [7] Vaswani, A., et al., “Attention is All You Need”, *NeurIPS*, 2017.
- [8] Devlin, J., et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *arXiv:1810.04805*, 2018.
- [9] Apache Software Foundation, “Apache Airflow Documentation”, <https://airflow.apache.org/docs/>, 2024.
- [10] Shekhawat, S., “MTech4SEM Project Repository”, <https://github.com/suryashekhawat/MTech4SEM>, 2026.